

ANDROID STUDIO

KOTLIN-BILDUMAK: MAPAK

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Lanerako tresna egokia.
2. Datuen elkarteak
3. Mapen oinarriak
 1. Mapak sortu
 2. Balioa bilatu
 3. Mapa aldatu
4. Mapen eragiketak
5. Mapa bat sortzea zerrenda batetik

LANERAKO TRESNA EGOKIA

"Lanerako tresna egokia erabili behar da," horixe erakutsi zion Andrews jaunak bere seme Jim-i, aitaren moduan esku-trebe bihurtzen hasi berri baitzen. "Clavo bat baduzu, Martilloa erabili behar duzu, ez bihurkina."

Jim-i tresna egokia aukeratzen laguntzeko, bere tresna-kutxan zituen tresna eta osagarri ezberdinen taula bat marraztu zuen.

MULTZOETARAKO SARREERA

Hardware	Tool to Use
 Nail	 Hammer
 Hex Nut	 Wrench
 Hex Bolt	 Wrench
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver

LANERAKO TRESNA EGOKIA

Taula hauek erakusten dute gauzen artean **loturak** daudela: Clavo bat Martillo batekin lotzen da, hexagono bat giltza batekin lotzen da, eta abar. Kapitulu honetan, horrelako taula baten Kotlin baliokidea sortuko dugu, baina hori egin aurretik, has gaitezen elkarte bakar bat sortzen.

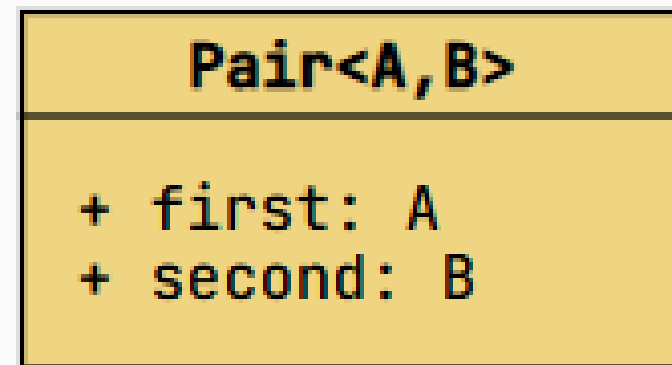
DATUEN ELKARTEA

Balio bi elkartzeko modu erraza da **Pair** izeneko klase bat erabiltzea. Klase honen eraikitzaileak bi parametro ditu. Eraikitzailea edozein bi objektuekin deitu ahal duzu, haien mota edozein dela ere. Gure kasuan, bi **String** objektu lotuko ditugu: bata Clavo batentzat eta bestea Martillo batentzat.

```
val association = Pair("Nail", "Hammer")
```

DATUEN ELKARTEA

Pair klasea oso sinplea da eta bi propietate ditu, **first** eta **second**, balioak berreskuratzeko erabil ditzakezunak. Jarraian, **Pair** klasea erakusten duen UML diagrama bat dago. **First** eta **second** motak eraikitzaileari ematen dizkiozun argumentuen moten arabera dira; beraz, diagrama honetan, **A** eta **B** benetako moten lekuko gisa erabiliko ditugu.



DATUEN ELKARTEA

First propietatea eraikitzaileari lehenengo parametro gisa eman zeniotena izango da, eta second bigarren parametro gisa eman zeniotena.

```
println(association.first) /  
println(association.second) /
```

Erraza, ezta?

DATUEN ELKARTEA

Orain, **Pair** klasearen eraikitzaileari deitu beharrean, batzuetan naturalagoa da **to()** izeneko funtzio bat erabiltzea, eta horrek zure ordezt **Pair** eraikitzailea deituko du. Funtzio hau edozein objektutan dei daiteke. Gure kodea **to()** funtzioa erabiliz eguneratuko dugu.

```
val association = "Nail".to("Hammer")
```

DATUEN ELKARTEA

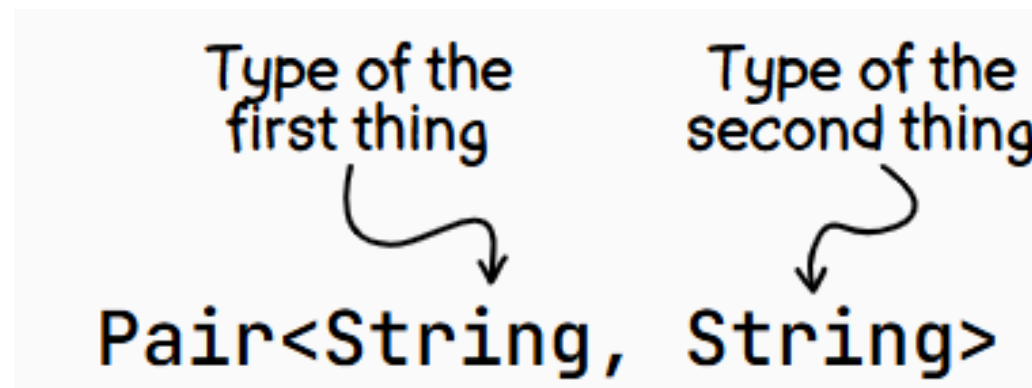
`To()` funtzioak puntuaziorik gabe erabiltzeko ezaugarri berezi bat ere badu. Beraz, honela ere sor dezakezu `Pair`:

```
val association = "Nail" to "Hammer"
```

Funtzio bati modu honetan dei egitea ahalbidetzen bazaio, funtzio infijo bat dela esaten da. Ez ditugu funtzio infijioak askotan ikusiko, baina garrantzitsua da badirela jakitea, kode mota hau ikusten duzunean nahastu ez zaitezten.

DATUEN ELKARTEA

Orain arte, tipoak inferitzea erabili dugu **association** aldagaiko tiporik ez idazteko. Aurreko kapituluan **List** eta **Set** bezala, **Pair** aldagai baten tipoa dituen gauzen motaren araberakoa da. Bai “Clavo” bai “Martillo” **String** motakoak direnez, **association** aldagaiko tipoa **Pair<String, String>** da.



DATUEN ELKARTEA

Eta, noski, tipoa modu esplizituan adieraz dezakegu honela:

```
val association: Pair<String, String> = "Nail" to "Hammer"
```

Orain elkarte bakarra arrakastaz egin dugunez, gauza bera egin dezakegu gainerako tresnekin... eta gero guztiak mapa batean jarri!

MAPEN OINARRIAK

Hona hemen berriro Andrews jauna-ren mahaia.

Hardware	Tool to Use
 Nail	 Hammer
 Hex Nut	 Wrench
 Hex Bolt	 Wrench
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver

MAPEN OINARRIAK

Kotlin-en, taula horrelako bat mapa deitzen da. Agian kaleen mapak edo altxorren mapak ezagutzen dituzu, baina ez hori da hemen hitz egiten ari garena. Terminoa matematikako mundutik dator, non mapa batek multzoen elementuen arteko harreman bat definitzen duen. Modu berean, Kotlin-en mapak ezkerreko zutabeko elementu bakoitzaren eta eskuineko zutabeko elementu dagokienaren arteko elkarte bat definitzen dute.

MAPEN OINARRIAK

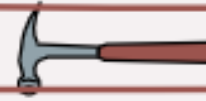
Lehenengo mapa sortu baino lehen, zenbait termino garrantzitsu landu ditzagun:

- Taularen ezkerreko zutabeko elementuak gako deitzen dira.
- Taularen eskuineko zutabeko elementuak balio deitzen dira.
- Gako eta balio baten elkartea mapa batean mapa sarrera deitzen da.

MAPEN OINARRIAK

Hardware		Tool to Use	
Keys	 Nail	 Hammer	Values
	 Hex Nut	 Wrench	
	 Hex Bolt	 Wrench	
	 Slotted Screw	 Slotted Screwdriver	
	 Phillips Screw	 Phillips Screwdriver	

MAPEN OINARRIAK

		Hardware	Tool to Use
Entries		Nail	 Hammer
		Hex Nut	 Wrench
		Hex Bolt	 Wrench
		Slotted Screw	 Slotted Screwdriver
		Phillips Screw	 Phillips Screwdriver

MAPEN OINARRIAK

Kontuan hartu beharreko arau garrantzitsu bat dago: mapa bateko gako bakoitza bakarra da. Hala ere, balioak bikoiztu egin daitezke. Bestela esanda, ezkerreko zutabean ezin da elementu bikoizturik egon, baina bai eskuineko zutabean. Beheko taulan ikus dezakezu hori. Ezkerreko zutabeko elementuak bakarrak dira, baina giltza ingelesa bi aldiz agertzen da eskuineko zutabean.

MAPEN OINARRIAK

No duplicates
in this column

Duplicates allowed
in this column

Hardware	Tool to Use
 Nail	 Hammer
 Hex Nut	 Wrench
 Hex Bolt	 Wrench
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver

Duplicated
value!

MAPA BAT SORTZEA MAPOF() ERABILIZ

Kontzeptu nagusiak ulertu ondoren, orain gure lehen mapa sortzeko garaia iritsi da! Horretarako, `mapOf()` funtzioa erabiliko dugu, `Pair` elkarte bat igorritz mapa honetara gehitu nahi dugun bakoitza.

```
val toolbox = mapOf(  
    "Nail" to "Hammer",  
    "Hex Nut" to "Wrench",  
    "Hex Bolt" to "Wrench",  
    "Slotted Screw" to "Slotted Screwdriver",  
    "Phillips Screw" to "Phillips Screwdriver",  
)
```

MAPA BAT SORTZEA MAPOF() ERABILIZ

Ohartuko zara hori listOf() eta setOf() funtzioei antzekoa dela, salbu eta hemen elementu guztiek bi parte dituzte: gakoa eta balioa, Pair-an batzen direnak. Bide batez...

Gogoratu: Pair bat sortzeko beste modu bat besterik ez duzu ukitzen. Era honetan ere sor genezakeen mapa:

```
val toolbox = mapOf(  
    Pair("Nail", "Hammer"),  
    Pair("Hex Nut", "Wrench"),  
    // ... and so on ...  
)
```

MAPA BAT SORTZEA MAPOF() ERABILIZ

Kotlin-en mapa baten eta Andrews jauna-ren taularen antzekotasunak ikus ditzakezu hauek elkarrekin jarritz:

```
val toolbox = mapOf(  
    "Nail" to "Hammer",  
    "Hex Nut" to "Wrench",  
    "Hex Bolt" to "Wrench",  
    "Slotted Screw" to "Slotted Screwdriver",  
    "Phillips Screw" to "Phillips Screwdriver",  
)
```

Hardware	Tool to Use
 Nail	 Hammer
 Hex Nut	 Wrench
 Hex Bolt	 Wrench
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver

MAPA BAT SORTZEA MAPOF() ERABILIZ

Listak, multzoak eta beste aldagai batzuk bezala, `println()` erabiliz mapa baten edukia inprimatu dezakezu.

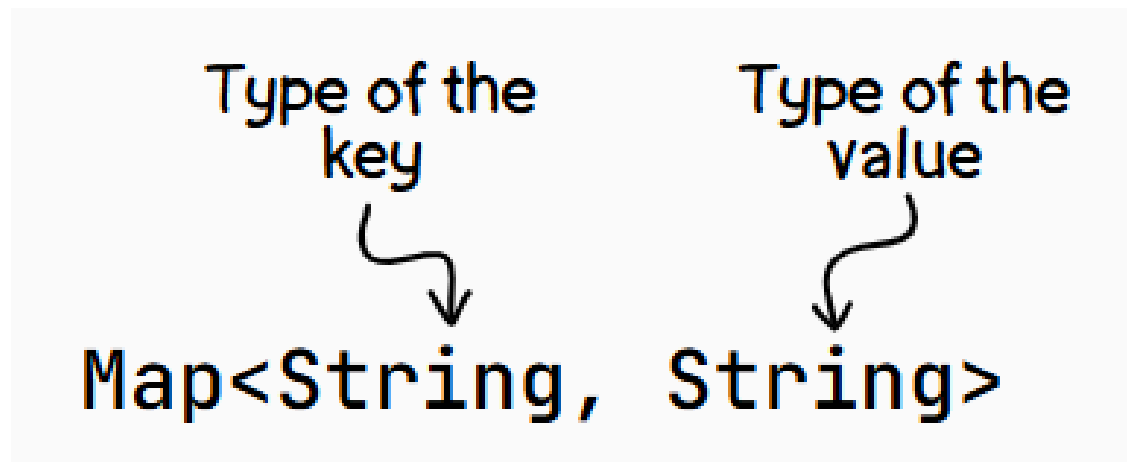
```
println(toolbox)
```

Inprimatzean, mapa sarrerek hargailu artean agertuko dira. Gakoak berdin izango dira = seinaleen ezkerrean eta balioak eskuinean.

```
{Nail=Hammer, Hex Nut=Wrench, Hex Bolt=Wrench, Slotted Screw=Slotted Screwdriver,
```

MAPA BAT SORTZEA MAPOF() ERABILIZ

Pair-en antzera, **Map** aldagai baten tipoa gakoaren eta balioaren motaren araberakoa da.



MAPA BAT SORTZEA MAPOF() ERABILIZ

Beraz, tipoa modu esplizituan honela idatzi dezakegu:

```
val toolbox: Map<String, String> = mapOf(  
    "Nail" to "Hammer",  
    "Hex Nut" to "Wrench",  
    "Hex Bolt" to "Wrench",  
    "Slotted Screw" to "Slotted Screwdriver",  
    "Phillips Screw" to "Phillips Screwdriver",  
)
```

BALIOA BILATZEN

Maiekin egin beharko duzun ohikoena balio bat bilatzea izango da. Adibidez, Jim-ek klabetak bat badu, zein tresna erabili behar duen bilatu behar du. Modu berean, Jim-ek klabetak ezkerreko zutabearen bilatuko luke eta ondoan dagoen tresna aurkituko luke; Kotlin-ek balio hori eman diezazuke gako bat ematen diozunean. Hori egiteko, `get()` funtzioa erabil dezakezu.

```
val tool = toolbox.get("Nail")  
println(tool) // Hammer
```

BALIOA BILATZEN

Zerrendetan bezala, baita indize bidezko sarrera-operadorea ere erabil dezakezu mapetan balio bat lortzeko.

```
val tool = toolbox["Nail"]  
println(tool) // Hammer
```

Get() funtzioa erabiltzean (edo indize bidezko sarrera-operadorea) mapa batean existitzen ez den gako batekin deitzen baduzu, balio bat null itzuliko du. Horrek esan nahi du **get()** funtzioak balioak nulua onartzen dituen mota bat itzultzen duela, balioak nulua onartzen ez dituen mota baten ordez.

BALIOA BILATZEN

Balio nuluren segurtasun-tresnak erabil ditzakezu (esaterako, elvis operadorea) balioak nulua onartzen ez duen mota batera itzultzeko. Bestela, `getValue()` erabil dezakezu `get()`-en ordeez. `GetValue()`-k balioak nulua onartzen ez duen mota bat itzuliko du, baina kontuz ibili: existitzen ez den gako bat ematen badiozu, errore-mezu bat ikusiko duzu eta zure kodeak exekutatzea utziko du.

BALIOA BILATZEN

```
val tool = toolbox.getValue("Nail")  
println(tool) // Hammer  
  
val anotherTool = toolbox.getValue("Wing Nut") // Error at runtime
```

Halaber, `getOrDefault()` erabil dezakezu gako bat existitzen ez bada balio lehenetsi bat emateko. Andrews jaunak ez badu tresnarik osagarri jakin baterako, eskuz konpondu beharko du!

```
val tool = toolbox.getOrDefault("Hanger Bolt", "Hand")
```

MAPA ALDATU

Beste bilduma motekin bezala, mapek ere mutagarritasun bi aldaera dituzte: **MutableMap** eta aldaezina den **Map**. Aldaera mutagarriak bere edukia aldatzen uzten dizu, eta aldaezinak mapa instantzia berri bat sortzea eskatzen du, eta hau aldagaia berri bati edo lehendik dagoen bati esleitu behar zaio.

Lehenik eta behin, **MutableMap** nola aldatu ikusi dezagun. Hasteko, **mutableMapOf()** erabili beharko dugu mapa sortzeko, **mapOf()** besterik ez ordeztu.

MAPA ALDATU

```
val toolbox = mutableMapOf(  
    "Nail" to "Hammer",  
    "Hex Nut" to "Wrench",  
    "Hex Bolt" to "Wrench",  
    "Slotted Screw" to "Slotted Screwdriver",  
    "Phillips Screw" to "Phillips Screwdriver",  
)
```

MAPA ALDATU

Sarrera berri bat gehitzeko, `put()` funtzioa erabil dezakezu, non lehenengo argudioa gakoa den eta bigarren argudioa balioa den.

```
toolbox.put("Lumber", "Saw")
```

`Get()` funtzioarekin bezala, ordea, Kotlin garatzaileek normalean sarrera-operadorea erabiltzen dute `put()` funtzioari zuzenean deitu beharrean.

```
toolbox["Lumber"] = "Saw"
```


MAPA ALDATU

Era berean, balio bat aldatu dezakezu modu berean. Lehendik dagoen gako bat besterik ez duzu eman behar.

```
toolbox["Hex Bolt"] = "Nut Driver"
```

Azkenik, sarrera bat ezabatu dezakezu `remove()` funtzioa erabiliz.

```
toolbox.remove("Lumber")
```

MAPA ALDATU

Kontuan izan balio bat alda dezakezun arren, gako bat ezin duzula aldatu. Horren ordeztu, gako bat ezabatu eta sarrera berri bat txerta dezakezu.

```
toolbox.remove("Phillips Screw")  
toolbox["Cross Recess Screw"] = "Phillips Screwdriver"
```

MAPA ALDATU: MAPA ALDAEZINAK

Zerrenda eta multzo aldaezinekin bezala, plus eta minus operadoreak erabil ditzakezu mapa aldaezin batean. Gogoratu hori egiterakoan mapa instantzia berri bat sortuko dela, eta hau normalean aldagaia bati esleituko zeniokeela. Hurrengo kodeak aurretik egindako eragiketa berberak erakusten ditu, baina mapa aldaezin batean. Kontuan izan hemen **var** hitz-gakoa erabiltzen dugula emaitza bakoitza **toolbox** aldagaian esleitzeko!

MAPA ALDATU: MAPA ALDÆEZINAK

```
var toolbox = mapOf(  
    "Nail" to "Hammer",  
    "Hex Nut" to "Wrench",  
    "Hex Bolt" to "Wrench",  
    "Slotted Screw" to "Slotted Screwdriver",  
    "Phillips Screw" to "Phillips Screwdriver",  
)  
  
// Add an entry  
toolbox = toolbox + Pair("Lumber", "Saw")  
  
// Update an entry  
toolbox = toolbox + Pair("Hex Bolt", "Nut Driver")  
  
// Remove an entry  
toolbox = toolbox - "Lumber"  
  
// Simulate changing a key  
toolbox = toolbox - "Phillips Screw"  
toolbox = toolbox + Pair("Cross Recess Screw", "Phillips Screwdriver")
```

MAPEN ERAGIKETAK

`List` eta `Set` objektuekin bezala, `Map` objektuek haien gainean egin daitezkeen eragiketak dituzte, eta horietako batzuk oso ezagunak egingo zaizkizu. Hasi gaitezen `forEach()` funtzioarekin.

`bakoitzarentzat()`

`ForEach()` funtzioa ia berdina da `List` eta `Set` objektuetan dagoenarekin. Lambda bat erabiltzen duzu mapa bakoitzeko sarrerarekin zerbait egiteko. Mapek sarrerak gordetzen dituztenez, lambda-ren parametroa `Map.Entry` motakoa izango da.

MAPEN ERAGIKETAK

`Map.Entry` oso antzekoa da kapitulu honetan ikusi dugun `Pair` klasearekin: bi propietate ditu, baina `first` eta `second` izenak izan beharrean, `key` eta `value` deitzen dira.

<code>Map.Entry<K, V></code>
+ <code>key</code>: K + <code>value</code>: V

MAPEN ERAGIKETAK

Hona hemen nola erabili dezakezun `forEach()` funtzioa `Map` batean.

```
toolbox.forEach { entry ->
    println("Use a ${entry.value} on a ${entry.key}")
}
```

Kode hau exekutatzean, honako hau ikusiko duzu.

```
Use a Hammer on a Nail
Use a Wrench on a Hex Nut
Use a Wrench on a Hex Bolt
Use a Slotted Screwdriver on a Slotted Screw
Use a Phillips Screwdriver on a Phillips Screw
```

MAPEN ERAGIKETAK

Hona hemen nola erabili dezakezun `forEach()` funtzioa `Map` batean.

```
toolbox.forEach { entry ->
    println("Use a ${entry.value} on a ${entry.key}")
}
```

Kode hau exekutatzean, honako hau ikusiko duzu.

```
Use a Hammer on a Nail
Use a Wrench on a Hex Nut
Use a Wrench on a Hex Bolt
Use a Slotted Screwdriver on a Slotted Screw
Use a Phillips Screwdriver on a Phillips Screw
```


MAPEN ERAGIKETAK

`forEach()` funtzioaren oso antzekoa denez, atal nagusiak identifikatzeko gai izan beharko zenuke. Hona hemen atal horiek:

Map variable

Lambda parameter

```
toolbox.forEach { entry →  
    println("${entry.key} and ${entry.value}")  
}
```

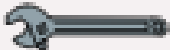
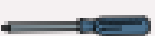
Code to run
for each entry

IRAGAZKETA

Zerrenda eta multzoekin antzera, **filter** erabil dezakezu mapa bat iragazteko. Kontuan izan zerrendekin ikusi genuen bezala, funtzio honek ez du dagoeneko dagoen mapa bat aldatzen, mapa instantzia berri bat sortzen du; beraz, emaitza aldagai bati esleitzea nahi izango duzu ziurrenik.

IRAGAZKETA

Tresna kaxa destorniladoreetara bakarrik iragaz dezagun.

Hardware	Tool to Use
 Nail	 Hammer
 Hex Nut	 Wrench
 Hex Bolt	 Wrench
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver



Hardware	Tool to Use
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver

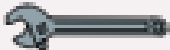
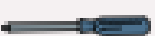
IRAGAZKETA

Tresna kaxa destorniladoreetara bakarrik iragaz dezagun.

```
val screwdrivers = toolbox.filter { entry ->  
    entry.value.contains("Screwdriver")  
}
```

IRAGAZKETA

Emaitza **Map** berri bat da, destorniladoreak soilik dituen edukiarekin.

Hardware	Tool to Use
 Nail	 Hammer
 Hex Nut	 Wrench
 Hex Bolt	 Wrench
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver



Hardware	Tool to Use
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver

IRAGAZKETA

Kasu honetan, *value* (balioa) bidez iragazi dugu, baina berdin erraz iragazi dezakezu gakoaren arabera.

```
val screwdrivers = toolbox.filter { entry ->
    entry.key.contains("Screw")
}
```

KARTOGRAFIA

Bai, `Map` bat mapa dezakezu! Erabili besterik ez duzu `mapKeys()` eta `mapValues()` funtzioak gakoak edo balioak eraldatzeko. Aurreko kapituluan ikusi genituen bilduma-eragiketekin bezala, eragiketa-kate bat sor dezakezu hemen ere. Mapa ditzagun bai gakoak bai balioak kate batean.

```
val newToolbox = toolbox
    .mapKeys { entry -> entry.key.replace("Hex", "Flange") }
    .mapValues { entry -> entry.value.replace("Wrench", "Ratchet") }
```

KARTOGRAFIA

Hardware	Tool to Use
 Nail	 Hammer
 Hex Nut	 Wrench
 Hex Bolt	 Wrench
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver



Hardware	Tool to Use
 Nail	 Hammer
 Flange Nut	 Ratchet
 Flange Bolt	 Ratchet
 Slotted Screw	 Slotted Screwdriver
 Phillips Screw	 Phillips Screwdriver

KARTOGRAFIA

Map objektuek beste eragiketa ugari dituzte, eta horiek Kotlin API dokumentazioan aurki ditzakezu. Hala ere, aurrera jarraitu aurretik beste eragiketa bat aztertuko dugu: **withDefault()**.

LEHENETSITAKO BALIOAK EZARTZEA

Lehenago ikusi dugun bezala, `getOrDefault()` erabil dezakezu gakoa ez dagoen kasuak dotore kudeatzeko. Hala ere, hau zailtasunez kontrola daiteke balio lehenetsi bera behin eta berriro erabiltzen baduzu...

```
val tool = toolbox.getOrDefault("Hanger Bolt", "Hand")  
val anotherTool = toolbox.getOrDefault("Dowel Screw", "Hand")  
val oneMoreTool = toolbox.getOrDefault("Eye Bolt", "Hand")
```

LEHENETSITAKO BALIOAK EZARTZEA

Lehenago ikusi dugun bezala, `getOrDefault()` erabil dezakezu gakoa ez dagoen kasuak dotore kudeatzeko. Hala ere, hau zailtasunez kontrola daiteke balio lehenetsi bera behin eta berriro erabiltzen baduzu...

```
val tool = toolbox.getOrDefault("Hanger Bolt", "Hand")  
val anotherTool = toolbox.getOrDefault("Dowel Screw", "Hand")  
val oneMoreTool = toolbox.getOrDefault("Eye Bolt", "Hand")
```

LEHENETSITAKO BALIOAK EZARTZEA

Horren orde, `withDefault()` izeneko eragiketa erabil dezakezu, eta horrek jatorrizkoan oinarritutako mapa berri bat itzuliko dizu. Mapa berri honetan, betiere `getValue()`-ri gako bat ematen badiozu existitzen ez dena, lambda bat exekutatu du eta emaitza itzuliko du. Honela ikusiko da:

```
toolbox = toolbox.withDefault { key -> "Hand" }
```

LEHENETSITAKO BALIOAK EZARTZEA

Orain, balio lehenetsia ematearen ordeztu balio bat lortu nahi duzun bakoitzean `getValue()` normal gisa deitu dezakezu. Hau oso onuragarria da, balio lehenetsia aldatu nahi baduzu, leku bakar batean egin dezakezulako aldaketa, eta ez toki askotan.

```
val tool = toolbox.getValue("Hanger Bolt")  
val anotherTool = toolbox.getValue("Dowel Screw")  
val oneMoreTool = toolbox.getValue("Eye Bolt")
```

LEHENETSITAKO BALIOAK EZARTZEA

Kontuan izan honek `getValue()`-rekin funtzionatzeko duela, baina ez `get()`-rekin edo sarrera-operadorearekin; horiek null itzultzen jarraituko dute gakoa aurkitzen ez bada.

Orain badakizu nola sortu eta aldatu mapak, nola atera balioak haietatik eta nola erabili bilduma-eragiketak haiekin. Baina gauzak benetan dibertigarri bihurtzen dira mapak beste bildumekin erabiltzen hasten zarenean. Azter dezagun hori jarraian.

MAPA BAT SORTZEA ZERREND BATETIK

Eskuz sortu ditugu mapak `mapOf()` funtzioa erabiliz. Baita ere posible da mapa berriak sortzea lehendik dauden zerrenda edo multzoetan oinarrituta. Funtzio garrantzitsu batzuk erabiliz, zure datuak hainbat modu ezberdinetan zatitu eta segmentatu ditzakezu. Horretarako, noski, hasteko zerrenda bat beharko dugu.

Mr. Andrews-en tresna-kaxako tresnak adierazteko `String` sinple bat erabili beharrean, sortu dezagun klase bat, tresnaren izena, haren pisua ontzatan eta erabiltzen duen hardware mota gorde ahal izateko.

MAPA BAT SORTZEA ZERRENDATUTAKO

Mr. Andrews-en tresna-kaxako tresnak adierazteko **String** sinple bat erabili beharrean, sortu dezagun klase bat, tresnaren izena, haren pisua ontzatan eta erabiltzen duen hardware mota gorde ahal izateko.

```
class Tool(  
    val name: String,  
    val weightInOunces: Int,  
    val correspondingHardware: String,  
)
```


MAPA BAT SORTZEA ZERRENDATETIK

Orain, tresna-kaxako tresnak barne hartzen dituen Tool objektuen zerrenda bat sortuko dugu, Mr. Andrews-en tresnak barne hartuko dituenak.

```
val tools = listOf(  
    Tool("Hammer", 14, "Nail"),  
    Tool("Wrench", 8, "Hex Nut"),  
    Tool("Wrench", 8, "Hex Bolt"),  
    Tool("Slotted Screwdriver", 5, "Slotted Screw"),  
    Tool("Phillips Screwdriver", 5, "Phillips Screw"),  
)
```

Orain zerrenda bat dugunez, prest gaude horretatik mapa batzuk sortzeko!

MAPA BAT SORTZEA ZERRENDATETIK

Orain, tresna-kaxako tresnak barne hartzen dituen Tool objektuen zerrenda bat sortuko dugu, Mr. Andrews-en tresnak barne hartuko dituenak.

```
val tools = listOf(  
    Tool("Hammer", 14, "Nail"),  
    Tool("Wrench", 8, "Hex Nut"),  
    Tool("Wrench", 8, "Hex Bolt"),  
    Tool("Slotted Screwdriver", 5, "Slotted Screw"),  
    Tool("Phillips Screwdriver", 5, "Phillips Screw"),  
)
```

Orain zerrenda bat dugunez, prest gaude horretatik mapa batzuk sortzeko!

OBJEKTU-ZERRENDAREN BATEN PROPIETATEEN ELKARTEA

Objektu-zerrenda batetik mapa bat sortzeko `associate()` funtzioa erabil dezakezu.

Hasteko, erabil dezagun `associate()`

```
val toolbox = tools.associate { tool ->  
    tool.correspondingHardware to tool.name  
}
```

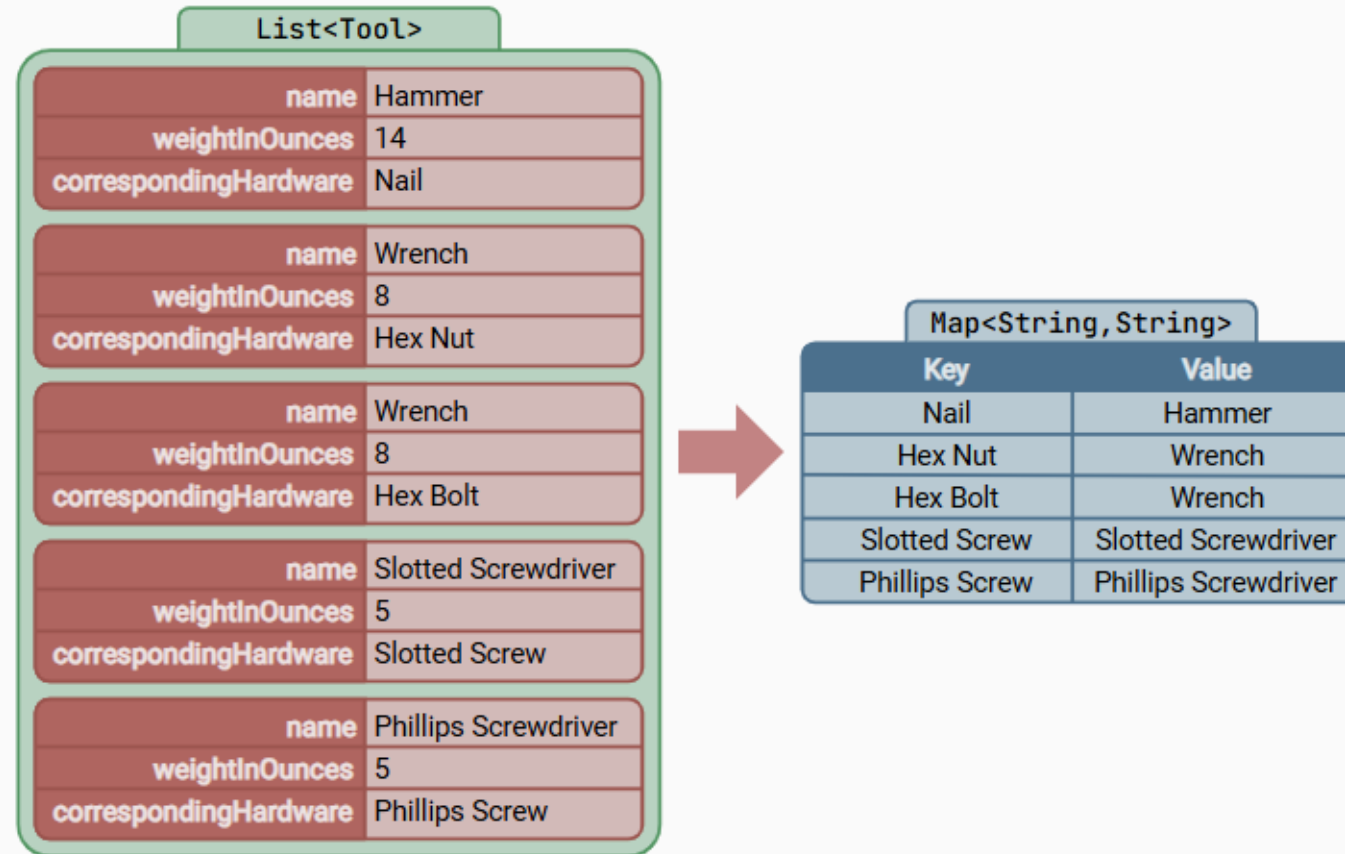
OBJEKTU-ZERRENDAREN BATEN PROPIETATEEN ELKARTEA

Espero dut une honetarako bilduma-eragiketekin gero eta erosoago sentitzea. Zerrendako elementu bakoitzarentzat, `associate()` funtzioak berari esleitutako lambda-adierazpena exekutatuko du. Lambda-adierazpenak `Pair` bat itzultzen du, emaitza-mapako gakoa eta balioa dituenena.

OBJEKTU-ZERRENDAREN BATEN PROPIETATEEN ELKARTEA

Espero dut une honetarako bilduma-eragiketekin gero eta erosoago sentitzea. Zerrendako elementu bakoitzarentzat, `associate()` funtzioak berari esleitutako lambda-adierazpena exekutatuko du. Lambda-adierazpenak `Pair` bat itzultzen du, emaitza-mapako gakoa eta balioa dituenena.

OBJEKTU-ZERRENTA BATEN PROPIETATEEN ELKARTEA



OBJEKTU-ZERRENDAA BATEN PROPIETATEEN ELKARTEA

Hona hemen `associate()` funtzioaren azalpena.

```
val toolbox = tools.associate { tool ->
    tool.correspondingHardware to tool.name
}
```

New map variable Original list variable Lambda parameter

Pair to insert as an entry in the new map.

OBJEKTU-ZERRENDA BATEN PROPIETATEEN ELKARTEA

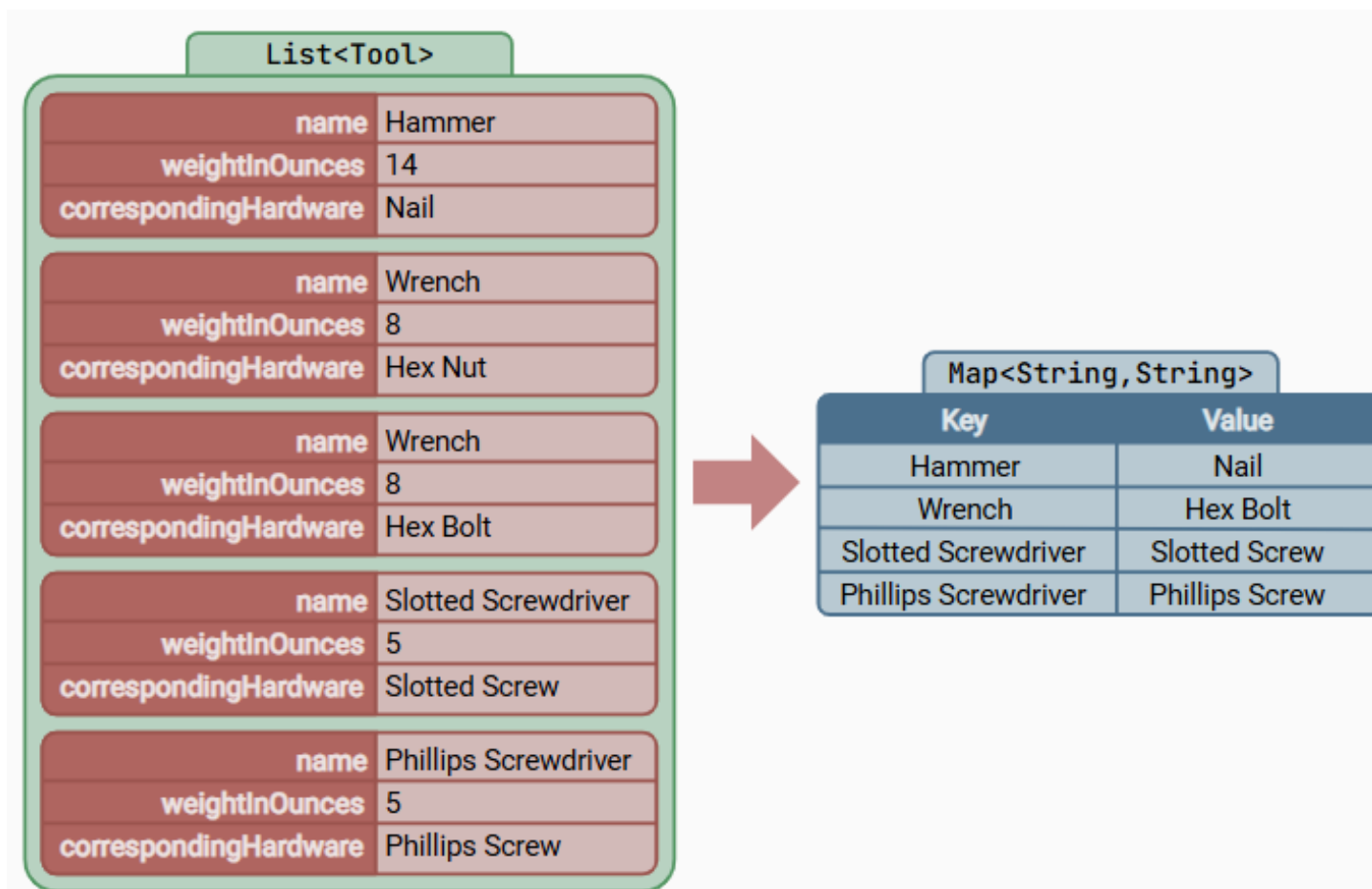
Maiz, mapa emaitzan dauden elementuen kopurua jatorrizko zerrendako elementu-kopuru berbera izango da. Kasu batzuetan, gutxiago izan daitezke. Mapan gakoak bakarrak direnez, lehendik dagoen gako bat gehitzen saiatzen bazara, balio berriak aurrekoa gainidatziko du.

```
val toolbox = tools.associate { tool ->  
    tool.name to tool.correspondingHardware  
}
```


OBJEKTU-ZERRENDAREN BATEN PROPIETATEEN ELKARTEA

Jatorrizko zerrendak bi **Tool** objektu ditu, **Wrench** izenarekin. Beraz, **associate()** funtzioak lehenengoarekin topo egiten duenean, mapara gehitzen du, baina bigarrenarekin topo egiten duenean, lehen balioa ordezkatzeko du. Hori dela eta, emaitza-mapak **Hex Bolt** soilik jasotzen du, **Hex Nut** ordezkari, azkenak **Hex Bolt** izanik.

OBJEKTU-ZERRENTA BATEN PROPIETATEEN ELKARTEA



OBJEKTU-ZERRENDAREN BATEAN PROPIETATEEN ELKARTEA

Kasu honetan, jatorrizko zerrendako elementuak baino sarrera gutxiago daude mapan. Hau da, 4 sarrera baino ez daude mapan, zerrendako 5 elementurekin alderatuta.

ELKARTEAREN BESTE FUNTZIO BATZUK

associate() funtzioaren beste aldaera batzuk ere badira ezagutzea komeni direnak. Bereziki erabilgarriak dira jatorrizko zerrendako elementua emaitza-mapako gakoa edo balioa izan dadin nahi baduzu.

Adibidez, tresnen izenak gisa dituen mapa bat sortu nahi baduzu eta balioa Tool objektua izan dadin, *associateBy()* erabil dezakezu. Funtzio honen lambdak gakoa soilik itzultzen du. Jatorrizko zerrendako elementua izango da balioa.

```
val toolsByName = tools.associateBy { tool -> tool.name }
```

ELKARTEAREN BESTE FUNTZIO BATZUK

List<Tool>	
name	Hammer
weightInOunces	14
correspondingHardware	Nail
name	Wrench
weightInOunces	8
correspondingHardware	Hex Nut
name	Wrench
weightInOunces	8
correspondingHardware	Hex Bolt
name	Slotted Screwdriver
weightInOunces	5
correspondingHardware	Slotted Screw
name	Phillips Screwdriver
weightInOunces	5
correspondingHardware	Phillips Screw



Map<String, Tool>							
Key	Value						
Hammer	<table><tr><td>name</td><td>Hammer</td></tr><tr><td>weightInOunces</td><td>14</td></tr><tr><td>correspondingHardware</td><td>Nail</td></tr></table>	name	Hammer	weightInOunces	14	correspondingHardware	Nail
name	Hammer						
weightInOunces	14						
correspondingHardware	Nail						
Wrench	<table><tr><td>name</td><td>Wrench</td></tr><tr><td>weightInOunces</td><td>8</td></tr><tr><td>correspondingHardware</td><td>Hex Bolt</td></tr></table>	name	Wrench	weightInOunces	8	correspondingHardware	Hex Bolt
name	Wrench						
weightInOunces	8						
correspondingHardware	Hex Bolt						
Slotted Screwdriver	<table><tr><td>name</td><td>Slotted Screwdriver</td></tr><tr><td>weightInOunces</td><td>5</td></tr><tr><td>correspondingHardware</td><td>Slotted Screw</td></tr></table>	name	Slotted Screwdriver	weightInOunces	5	correspondingHardware	Slotted Screw
name	Slotted Screwdriver						
weightInOunces	5						
correspondingHardware	Slotted Screw						
Phillips Screwdriver	<table><tr><td>name</td><td>Phillips Screwdriver</td></tr><tr><td>weightInOunces</td><td>5</td></tr><tr><td>correspondingHardware</td><td>Phillips Screw</td></tr></table>	name	Phillips Screwdriver	weightInOunces	5	correspondingHardware	Phillips Screw
name	Phillips Screwdriver						
weightInOunces	5						
correspondingHardware	Phillips Screw						

ELKARTEAREN BESTE FUNTZIO BATZUK

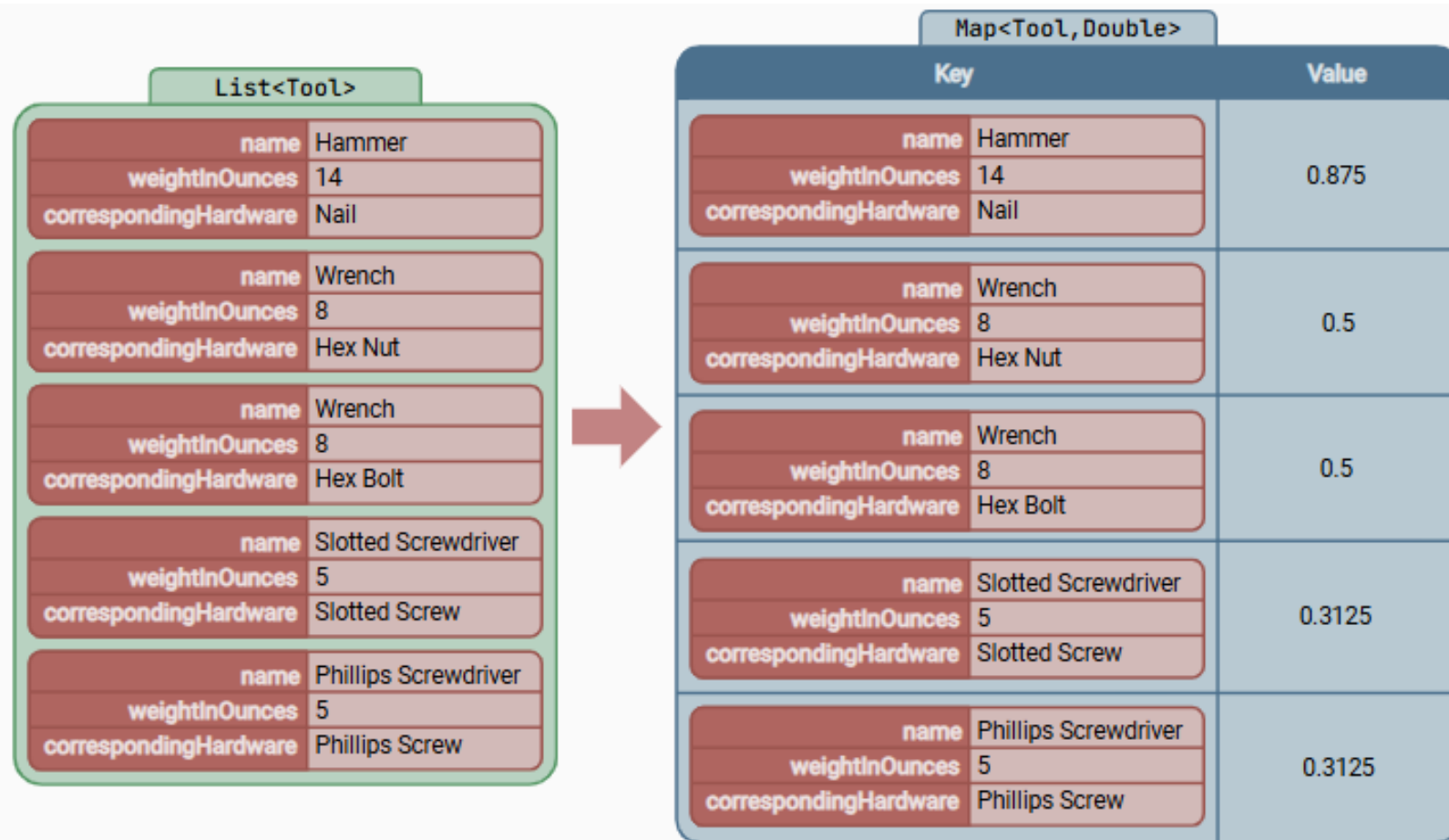
Mapa honekin, erraz aurki dezakezu tresna bat bere izenaren arabera!

```
val hammer = toolsByName["Hammer"]
```

Alderantziz, Tool objektua gako eta lambdan zehaztutako balioa duen mapa bat sortu nahi baduzu, `associateWith()` funtzioa erabil dezakezu. Funtzio honen lambdak balioa itzultzen du, eta jatorrizko zerrendako elementua izango da gakoa.

ELKARTEAREN BESTE FUNTZIO BATZUK

```
val toolWeightInPounds = tools.associateWith { tool ->
    tool.weightInOunces * 0.0625
}
```



ELKARTEAREN BESTE FUNTZIO BATZUK

Martillo baten pisua lortzeko, aldez aurretik martillo izeneko objektu bat beharko zenuke.

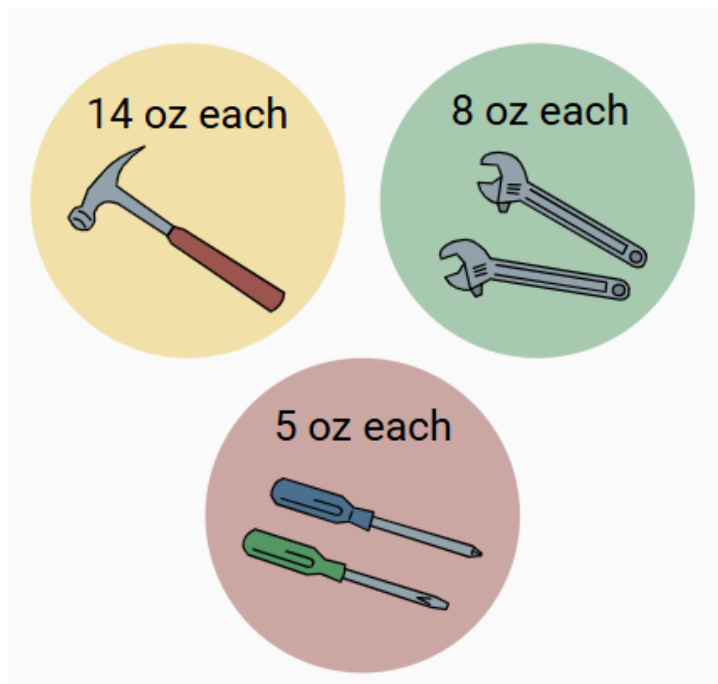
```
val hammerWeightInPounds = toolWeightInPounds[hammer]
```


ZERRENDĀ BATEN ELEMENTUĀK

ZERRENDĀ-MAPĀ BATEAN

TALDEKATZEA

Batzuetan, zerrenda bat duzunean, hainbat zerrenda txikitan banatu nahi izaten duzu ezaugarri jakin baten arabera. Adibidez, tresna-zerrenda (**tools** lista) har dezakegu eta pisuaren arabera bana dezakegu.



ZERRENDAK BATEAN ELEMENTUAK

ZERRENDAK-MAPAK BATEAN

TALDEKATZEA

Horretarako, `groupBy()` funtzioa erabil dezakegu. Funtzio honek eman zaion lambda-adierazpena exekutatu du zerrendako elementu bakoitzarentzat. Lambda-adierazpenak emaitza bera ematen duen elementuak zerrenda batean bilduko dira eta mapa batean txertatuko dira.

```
val toolsByWeight = tools.groupBy { tool ->
    tool.weightInOunces
}
```

ZERRENDAK BATEN ELEMENTUAK

ZERRENDAK-MAPAK BATEAN

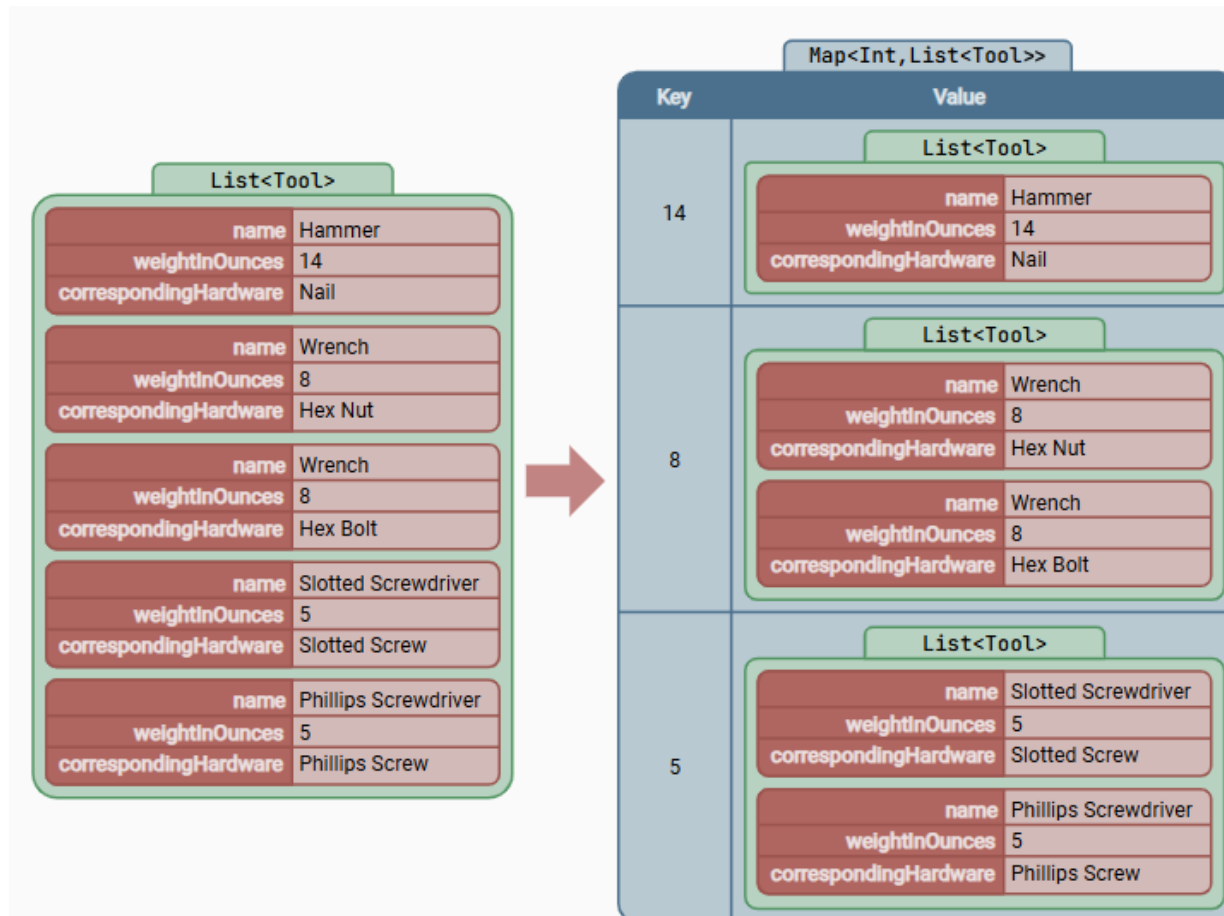
TALDEKATZEA

Emaitza **Map** bat da, 14 ontzako pisua duten tresna-zerrenda batekin, 8 ontzako pisua duten tresna-zerrenda batekin eta 5 ontzako pisua duten tresna-zerrenda batekin. Mapako gakoa ontzatan pisua da eta mapako balioa pisu bera duten tresnen zerrenda da.

ZERRENDÄ BATEN ELEMENTUAK

ZERRENDÄ-MAPA BATEAN

TALDEKATZEA



ZERRENDÄ BATEN ELEMENTUAK

ZERRENDÄ-MAPA BATEAN

TALDEKATZEA

Hona hemen `groupBy()` funtzioaren azalpen bat:

The diagram shows the following code snippet with annotations:

```
val toolsByWeight = tools.groupBy { tool →  
    tool.weightInOunces  
}
```

Annotations:

- New map variable**: Points to `toolsByWeight`.
- Original list variable**: Points to `tools`.
- Lambda parameter**: Points to `tool`.
- Elements that return the same thing here will be grouped together**: Points to the lambda body `tool.weightInOunces`.

ZERRENDAKO BATEAN ELEMENTUAK

ZERRENDAKO-MAPAKO BATEAN

TALDEKATZEAK

Jatorrizko zerrendako elementuaren ordean emaitza-zerrendetan beste zerbaite nahi baduzu, bigarren argumentu bat erabiliz dei diezaiokizu funtzio honi. Horretarako, esleitu lambda bat emaitza-zerrendan nahi duzun hori itzultzen duena. Adibidez, zerrenda horietan tresnen izenak soilik nahi badituzu, honela egin dezakezu:

```
val toolNamesByWeight = tools.groupBy(  
    { tool -> tool.weightInOunces },  
    { tool -> tool.name }  
)
```

ZERRENDÄ BATEN ELEMENTUAK

ZERRENDÄ-MAPA BATEAN

TALDEKATZEA

